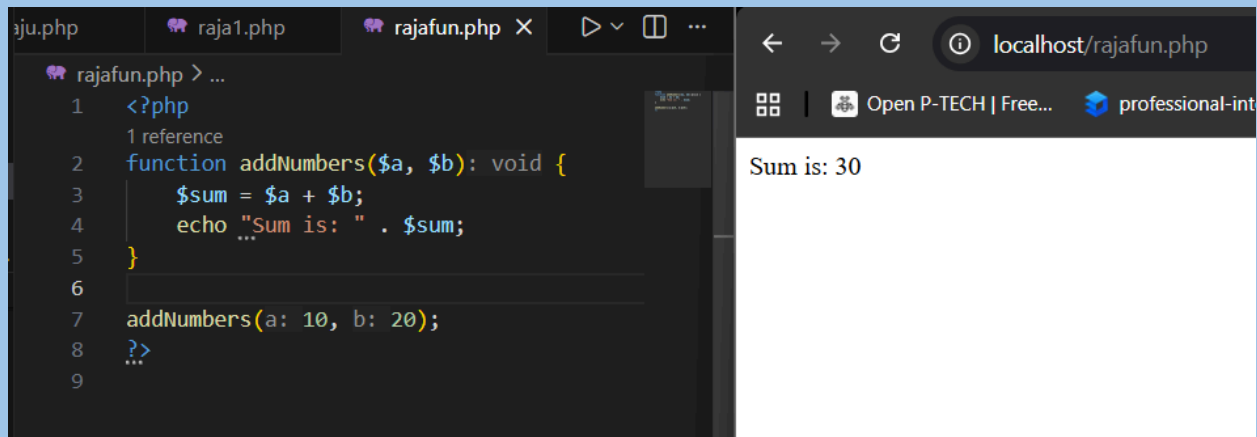


1. Function Parameters (Arguments & Types)

Definition

Parameters (also called arguments) are variables that receive values when a function is called. They allow functions to be flexible and reusable.



Style	Syntax example	Main use-case	Security note
By value (default)	<code>function add(\$a, \$b)</code>	Normal calculations	Safe – original not changed
By reference	<code>function increment(&\$count)</code>	Modify original variable	Dangerous if not intended
Default value	<code>function greet(\$name = "Guest")</code>	Optional arguments	Can hide logic bugs

Type declaration	<code>function save(int \$id, string \$name)</code>	Prevent wrong data types	Helps catch bugs early
Nullable type	<code>function findUser(?int \$id)</code>	Optional database row	Must check for null
Union types (PHP 8+)	<code>function setLimit(int float \$limit)</code>	Accept multiple types	Cleaner than old type juggling
Variadic (splat)	<code>function logAll(string ...\$messages)</code>	Variable number of arguments	Remember to sanitize each one
Named arguments (PHP 8+)	<code>connect(user: "root", pass: "toor")</code>	Skip order, improve readability	Very safe & self-documenting

2. Arrays & Multidimensional Arrays

Definition

Arrays store multiple values in a single variable. PHP arrays are ordered maps (key → value).

Types

- Indexed: `$arr = [10, 20, 30];` → keys are 0,1,2,...
- Associative: `$user = ['name' => 'Raju', 'role' => 'pentester'];`
- Multidimensional: arrays inside arrays (like tables, configs, JSON-like structures)

Important Array Functions (Security Context)

- `count()`, `sizeof()` → length
- `array_key_exists()`, `isset()` → check keys safely

- `in_array()` → search value
- `array_merge()`, `array_replace()` → combine
- `array_filter()`, `array_map()`, `array_walk()` → transform
- `explode()`, `implode()` → string ↔ array
- `json_decode($str, true)` → string → associative array

Example

The top screenshot displays a web browser at `localhost/array.php` showing a "Fruits List:" with the following items: Apple, Banana, Mango, and Orange. The corresponding PHP code in `array.php` is as follows:

```

1 <?php
2 // Creating a simple array
3 $fruits = array("Apple", "Banana", "Mango", "Orange");
4
5 // Displaying array elements using foreach loop
6 echo "<h3>Fruits List:</h3>";
7
8 foreach ($fruits as $fruit) {
9     echo $fruit . "<br>";
10 }
11 ?>
12

```

The bottom screenshot displays the same browser at `localhost/array.php` showing "Employee Details:" with the following information:

Name: Raju
Department: IT
Salary: ₹45000

Name: Anjali
Department: HR
Salary: ₹38000

Name: Suresh
Department: Finance
Salary: ₹52000

The corresponding PHP code in `array.php` is as follows:

```

1 <?php
2 // Creating a multidimensional array
3 $students = array(
4     array("Raju", "IT", 45000),
5     array("Anjali", "HR", 38000),
6     array("Suresh", "Finance", 52000)
7 );
8
9 // Displaying multidimensional array
10 echo "<h3>Employee Details:</h3>";
11
12 foreach ($students as $student) {
13     echo "Name: " . $student[0] . "<br>";
14     echo "Department: " . $student[1] . "<br>";
15     echo "Salary: ₹" . $student[2] . "<br><br>";
16 }
17 ?>
18

```

3. Most Important PHP Functions

Top functions grouped by usage in security/ethical hacking context (based on modern usage patterns):

Security & Input Handling (must-know)

- htmlspecialchars(), htmlentities() → XSS prevention
- strip_tags() → remove HTML/PHP tags
- trim(), ltrim(), rtrim() → clean input
- filter_var() / filter_input() → validate & sanitize (FILTER_SANITIZE_EMAIL, FILTER_VALIDATE_IP, etc.)
- password_hash(), password_verify() → secure passwords
- hash(), hash_hmac() → hashing, integrity checks

String Manipulation

- strlen(), mb_strlen() → length (use mb_* for UTF-8)
- str_replace(), str_ireplace() → replace
- strpos(), stripos(), str_contains() (PHP 8+) → search
- explode(), implode(), implode() → split/join
- preg_match(), preg_replace() → regex (very powerful for payloads)

Array & Data

- count(), isset(), empty(), array_key_exists()
- in_array(), array_search()
- array_merge(), array_slice(), array_filter()

File & Network (recon/exploit dev)

- file_get_contents() → read files / URLs
- file_put_contents() → write
- curl_init(), curl_exec() → HTTP requests (better than file_get_contents for control)

Others

- json_encode(), json_decode()
- var_dump(), print_r() → debugging
- header(), setcookie() → headers & sessions
- session_start() → session management
- die(), exit() → stop execution

Functions examples:

The image shows a web browser on the left and a code editor on the right. The browser displays the output of a PHP script at localhost/functions.php. The code editor shows the source code of the same script, which demonstrates various PHP functions categorized into String, Array, Math, Date & Time, File, and User Defined functions.

String Functions

Length: 13
Uppercase: ADHITYA RAMAN
Lowercase: adhitya raman
Substring: Adhitya

Array Functions

Total elements: 4
Updated Array: Array ([0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 50)
30 exists in array

Math Functions

Max: 50
Min: 10
Round 12.6: 13
Random Number: 16

Date & Time Functions

Today: 28-01-2026
Timestamp: 1769575506

File Functions

File not found

User Defined Function

Hello, Adhitya Raman 🙌

```
1 <?php
2 // =====
3 // 1. String Functions
4 // =====
5 $name = "Adhitya Raman";
6
7 echo "<h3>String Functions</h3>";
8
9 // strlen() - length of string
10 echo "Length: " . strlen(string: $name) . "<br>";
11
12 // strtoupper() - convert to uppercase
13 echo "Uppercase: " . strtoupper(string: $name) . "<br>";
14
15 // strtolower() - convert to lowercase
16 echo "Lowercase: " . strtolower(string: $name) . "<br>";
17
18 // substr() - get part of string
19 echo "Substring: " . substr(string: $name, offset: 0, length: 7) . "<br>";
20
21
22 // =====
23 // 2. Array Functions
24 // =====
25 $numbers = array(10, 20, 30, 40);
26
27 echo "<h3>Array Functions</h3>";
28
29 // count() - number of elements
30 echo "Total elements: " . count(value: $numbers) . "<br>";
31
32 // array_push() - add element
33 array_push(array: &$numbers, values: 50);
34
35 // print_r() - display array
36 echo "Updated Array: ";
37 print_r(value: $numbers);
```

🐘 employee_salary.php

🐘 db_connection.php

🐘 array.php

🐘 functions.php

🐘 functions.php > ...

```
47 // 3. Math Functions
48 // =====
49 echo "<h3>Math Functions</h3>";
50
51 // max() and min()
52 echo "Max: " . max(value: $numbers) . "<br>";
53 echo "Min: " . min(value_array: $numbers) . "<br>";
54
55 // round()
56 echo "Round 12.6: " . round(num: 12.6) . "<br>";
57
58 // rand() - random number
59 echo "Random Number: " . rand(min: 1, max: 100) . "<br>";
60
61
62 // =====
63 // 4. Date & Time Functions
64 // =====
65 echo "<h3>Date & Time Functions</h3>";
66
67 // date() - current date
68 echo "Today: " . date(format: "d-m-Y") . "<br>";
69
70 // time() - current timestamp
71 echo "Timestamp: " . time() . "<br>";
72
73
74 // =====
75 // 5. File Functions (Read-only demo)
76 // =====
77 echo "<h3>File Functions</h3>";
78
79 $file = "demo.txt";
80
81 // file_exists()
82 if (file_exists(filename: $file)) {
83     echo "File exists<br>";
```